

Optimal Control of a Cart-Pendulum System

Project Report

Modelling, control design and simulation of an inverted pendulum on a cart

System: Cart-pendulum (inverted pendulum on a cart)
Objective: Modelling and optimal control design
Tools: MATLAB & Simulink
Team: Charbel Beaini & Antonio Zerbato
Professor: Marco Baglietto

Contents

1	Dynamic Model of the Cart-Pendulum	2
1.1	Equations of motion	2
1.2	Dynamics function	2
1.3	Parameters and initialisation	3
2	First Controller: Infinite-Horizon LQ Regulator	4
2.1	Linearisation about the upright equilibrium	4
2.2	Discretisation with a zero-order hold	4
2.3	LQ regulator design via Riccati	5
2.4	The Riccati recursion	5
2.5	Closed-Loop Simulation in Simulink	6
2.5.1	The plant block	7
2.5.2	Integrator and initial condition	7
2.5.3	Feedback gain	7
2.5.4	The CartPend visualisation block	8
2.5.5	Results	8
3	Second Controller: LQ with an Extended Kalman Filter	9
3.1	Overview of the changes	10
3.2	Inside the <code>cartpole_ekf</code> block	10
3.2.1	Tuning matrices	10
3.2.2	Prediction step	11
3.2.3	Update step	11
3.2.4	Linearising only the dynamics	11
3.3	Measurement noise	12
3.4	Closed-loop response	12
3.5	Filter consistency: the three-sigma bands	12
4	Final Controller: Swing-Up plus LQ + EKF	13
4.1	Overview of the changes	14
4.2	The finite-horizon swing-up problem	14
4.3	From single shooting to multiple shooting	15
4.4	Choosing the horizon and the sample time	15
4.5	The optimal swing-up trajectory	15
4.6	The catch-and-hold supervisor	16
4.7	Results of the full controller	16
4.8	Comparing the plan with the real run	17

1 Dynamic Model of the Cart-Pendulum

1.1 Equations of motion

We started the project by deriving the equations of motion of our system. The cart-pendulum is made of a cart of mass M that slides horizontally and a pendulum of mass m hinged on top of it, free to rotate in the vertical plane. The cart is driven by a horizontal force F , which is the only control input, and the pendulum is actuated only indirectly through the motion of the cart. We describe the system with four states: the cart position x , the cart velocity $\dot{x}$, the pendulum angle θ and the pendulum angular velocity $\dot{\theta}$. The angle is measured from the upright position, so $\theta = 0$ corresponds to the pendulum balanced vertically above the cart. The state vector and the input are

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix}, \quad u = F.$$

The meaning and units of every symbol of the state and input are collected in Table 1.

Table 1: States, input and parameters of the cart-pendulum model.

Symbol	Quantity	Description / units
x	Cart position	Horizontal position of the cart [m]
$\dot{x}$	Cart velocity	Horizontal velocity of the cart [m/s]
θ	Pendulum angle	Measured from the upright, $\theta = 0$ is vertical up, [rad]
$\dot{\theta}$	Angular velocity	Rate of change of θ [rad/s]
F	Control input	Horizontal force applied to the cart [N]

Writing $s = \sin \theta$ and $c = \cos \theta$ for compactness, the position and angle simply carry their own velocities, while the cart acceleration $\ddot{x}$ and the angular acceleration $\ddot{\theta}$ follow from the coupled dynamics. The four equations stacked together give the state derivative

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} \dot{x} \\ \frac{F - b\dot{x} + m l s \dot{\theta}^2 - m g c s}{M + m s^2} \\ \dot{\theta} \\ \frac{(M + m) g s - c (F - b\dot{x} + m l s \dot{\theta}^2)}{l (M + m s^2)} \end{bmatrix}.$$

The two acceleration equations share the term $F - b\dot{x} + m l s \dot{\theta}^2$, which gathers the applied force, the cart friction and the centrifugal contribution of the swinging pendulum. The denominator $M + m s^2$ comes from the coupling between the cart and the pendulum and depends on the configuration. Collecting the four scalar equations gives the state-space form $\dot{\mathbf{x}} = f(\mathbf{x}, u)$ that we use throughout the project.

1.2 Dynamics function

We implemented these equations in the file `cartpole_dynamics.m`. The function takes the current state, the input and the five parameters, and returns the state derivative $\dot{\mathbf{x}}$. We kept the

intermediate quantities (`den` for the denominator and `common` for the shared term) so that the code reads in the same way as the equations above.

```

cartpole_dynamics.m

function xdot = cartpole_dynamics(x, u, M, m, l, b, g)
% Continuous-time nonlinear cart-pole
%
% xdot = cartpole_dynamics(x, u, M, m, l, b, g)
%
% State : x = [ pos ; vel ; theta ; omega ]
%             pos: cart position           [m]
%             vel: cart velocity           [m/s]
%             theta: pendulum angle       [rad] (theta = 0 is upright)
%             omega: pendulum angular vel [rad/s]
% Input : u = F: horizontal force on cart [N]
% Params: M, m, l, b, g
%
    vel = x(2);
    th = x(3);
    om = x(4);
    F = u(1);
    s = sin(th); c = cos(th);
    den = M + m*s^2;
    common = F - b*vel + m*l*s*om^2;
% d(vel)/dt
    xacc = ( common - m*g*c*s ) / den;
% d(omega)/dt
    thacc = ( (M+m)*g*s - c*common ) / (l*den);
    xdot = [ vel ; xacc ; om ; thacc ];
end

```

1.3 Parameters and initialisation

The dynamics function on its own only describes the model. To run anything we need numerical values for the parameters and a starting condition, so we collect all of this in a separate file `cartpole_setup.m`.

The chosen values are listed in Table 2.

Table 2: Numerical values used in `cartpole_setup.m`.

Parameter	Value	Description
M	0.5 kg	Cart mass
m	0.2 kg	Pendulum mass
l	0.3 m	Link length
b	0.1 N/(m/s)	Cart friction coefficient
g	9.81 m/s ²	Gravity
T_s	0.01 s	Sample time
$\mathbf{x}_0$	Set based on controller	Initial state

At the end of the setup we wrap the continuous dynamics into an anonymous function `fc`, so that the rest of the code can call the plant as $\dot{\mathbf{x}} = f_c(\mathbf{x}, u)$ without passing the parameters every time. This handle is the object we will discretise in the next step.

With the model defined in `cartpole_dynamics.m` and the parameters and initial state set in `cartpole_setup.m`, we have a complete continuous-time description of the system ready to be discretised and controlled.

2 First Controller: Infinite-Horizon LQ Regulator

The first controller we designed is an infinite-horizon linear quadratic (LQ) regulator. For this first version we assume that the exact state is available, meaning the controller reads all four states directly. This is the simplest possible setting and lets us check that the regulator works before adding any estimation. Later we drop this assumption and reconstruct the velocities with an extended Kalman filter, but for now the full state is known.

The LQ regulator is built on a linear model, so the work splits into three steps that all live in the setup file: we linearise the nonlinear dynamics about the upright equilibrium, we discretise the resulting linear model, and we solve a discrete Riccati recursion to obtain the feedback gain. We go through each in turn.

2.1 Linearisation about the upright equilibrium

The equations of motion derived earlier are nonlinear, while LQ design needs a linear state-space model. We therefore linearise the dynamics about the upright equilibrium $\theta = 0$, $\dot{\theta} = 0$ with zero force, which is exactly the operating point the controller has to hold. Keeping only the first-order terms (so $\sin \theta \approx \theta$, $\cos \theta \approx 1$, and the $\dot{\theta}^2$ term is dropped) gives the continuous linear model

$$\dot{\mathbf{x}} = A_c \mathbf{x} + B_c u, \quad y = G \mathbf{x},$$

with state and input matrices

$$A_c = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{b}{M} & -\frac{mg}{M} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{b}{Ml} & \frac{(M+m)g}{Ml} & 0 \end{bmatrix}, \quad B_c = \begin{bmatrix} 0 \\ \frac{1}{M} \\ 0 \\ -\frac{1}{Ml} \end{bmatrix}.$$

The output matrix selects the two quantities we actually measure, the cart position and the pendulum angle (the pose):

$$G = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}.$$

The matrices A_c , B_c and G are defined in `cartpole_setup.m`.

2.2 Discretisation with a zero-order hold

The controller runs in discrete time at the sample time T_s , so we convert the continuous model into an equivalent discrete one. We use a zero-order hold (ZOH), which assumes the input is held constant between two samples, exactly the way a digital controller drives the plant through a hold device. Under this assumption the discrete model is

$$\mathbf{x}_{k+1} = F \mathbf{x}_k + W u_k,$$

with

$$F = e^{A_c T_s}, \quad W = \left(\int_0^{T_s} e^{A_c \tau} d\tau \right) B_c.$$

In MATLAB this is done in a single line:

cartpole_setup.m

```
sys_d = c2d(ss(Ac,Bc,eye(4),0), Ts, 'zoh');
```

`ss(Ac,Bc,eye(4),0)` wraps the continuous matrices into a state-space object whose output is the full state. The function `c2d` then converts this continuous object to discrete time at the sample time `Ts` with the `'zoh'` method, evaluating the matrix exponential and its integral for us instead of by hand. The discrete state and input matrices are read back as `sys_d.A` and `sys_d.B`.

One note on naming: in the discrete model we follow the code and call these matrices F and W . This F is the discrete state matrix and is not the force F of the original model; from here on the input is always written u .

2.3 LQ regulator design via Riccati

With a linear discrete model we can pose the infinite-horizon LQ problem: find the input that minimises

$$J = \sum_{k=0}^{\infty} \left(\mathbf{x}_k^\top V \mathbf{x}_k + u_k^\top P u_k \right),$$

where V weights the state error and P weights the control effort. The solution is a constant state-feedback law

$$u_k = -L \mathbf{x}_k,$$

so once L is computed offline the controller is just a single gain. This is convenient in Simulink, where the regulator is implemented as one Gain block applying $u = -L \mathbf{x}$.

We chose the weights $V = \text{diag}(10, 5, 100, 5)$ and $P = 0.05$. The largest weight sits on the pendulum angle, since keeping the pole upright is the main objective; the position and the two velocities are penalised more gently, and the small input weight P allows reasonably strong actuation. The gain is then computed by our own Riccati loop:

cartpole_setup.m (LQ design)

```
F = sys_d.A;      % discrete state matrix
W = sys_d.B;      % discrete input matrix
% Inf Horizon LQ regulator: constant gain Lbar (T_k -> Tbar)
% Designed offline -> in Simulink it is just one Gain block: u = -L x.
V = diag([10 5 100 5]); % state weight
P = 0.05;          % input weight
L = lqr_riccati(F, W, V, P, 1e-12, 10000); % = Lbar, u = -L x
```

This block also lives in `cartpole_setup.m`, so by the end of the setup the constant gain L is ready to be dropped into the Simulink Gain block.

2.4 The Riccati recursion

The gain is produced by the function `lqr_riccati`, which solves the discrete infinite-horizon LQ problem by iterating the Riccati recursion backwards until it settles. For the system $\mathbf{x}_{k+1} = F \mathbf{x}_k + W u_k$ and the cost above, the recursion is

$$L_k = \left(P + W^\top T_{k+1} W \right)^{-1} W^\top T_{k+1} F,$$

$$T_k = V + F^\top T_{k+1} F - F^\top T_{k+1} W L_k.$$

We start from $T_N = V$ and step backwards. As the horizon grows the matrix T_k converges to a constant $\bar{T}$ and the gain to a constant $\bar{L}$, which are exactly the infinite-horizon solution we want.

In practice we stop as soon as two successive T matrices differ by less than a tolerance, measured with the Frobenius norm, and then recompute the final gain from the converged $\bar{T}$. If the loop reaches the maximum number of iterations without settling, the function raises a warning and returns the last gain it had.

```
lqr_riccati.m

function [L, T] = lqr_riccati(F, W, V, P, tol, maxit)
%   LQR_RICCATI   Infinite-horizon discrete LQR
%
%   System :  $x_{k+1} = F x_k + W u_k$ 
%   Cost J :  $\sum_k (x_k' V x_k + u_k' P u_k)$ 
%   Gain   :  $u_k = -L x_k$  (returned L = Lbar)
%   T      : converged Riccati matrix (= Tbar)
%
%   Recursion until we converge:
%    $L_{k+1} = (P + W' T_{k+1} W)^{-1} W' T_{k+1} F$ 
%    $T_k = V + F' T_{k+1} F - F' T_{k+1} W L_{k+1}$ 
% start from  $T_N = V$ 
T = V;
for k = 1:maxit
    L = (P + W'*T*W) \ (W'*T*F); % gain using current T (= T_{k+1})
    Tnew = V + F'*T*F - F'*T*W*L; % one step back -> T_k
    if norm(Tnew - T, 'fro') < tol
        T = Tnew;
        L = (P + W'*T*W) \ (W'*T*F); % final gain from the converged T
        return;
    end
    T = Tnew;
end
warning('lqr_riccati:noConverge', ...
    'Riccati did not converge in %d iterations.', maxit);
L = (P + W'*T*W) \ (W'*T*F);
end
```

The two lines inside the loop are the recursion written above: the first builds the gain from the current T , the second steps T one sample back. The backslash $\backslash$ solves the small linear system instead of forming an explicit inverse, which is both faster and better conditioned. Once converged, L holds the constant feedback gain $\bar{L}$ used by the controller and T holds the corresponding Riccati matrix $\bar{T}$.

2.5 Closed-Loop Simulation in Simulink

Having designed the gain offline, we built the closed loop in Simulink to test it. The model integrates the true nonlinear dynamics, applies the LQ feedback, and drives a 3D visualisation of the cart and pendulum. The overall layout is shown in Figure 1.

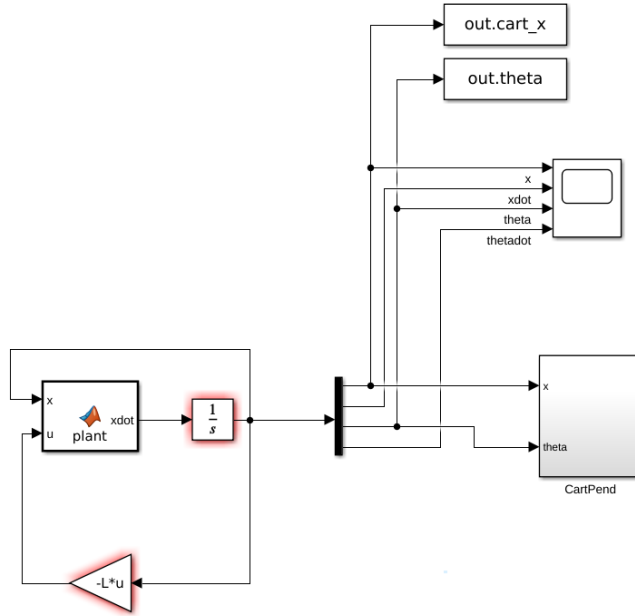


Figure 1: Simulink model of the closed loop: the **plant** block holds the nonlinear dynamics, the integrator turns $\dot{\mathbf{x}}$ into $\mathbf{x}$, the demux splits the state, and the $-L\mathbf{u}$ gain closes the loop. The state is logged and sent to the **CartPend** visualisation.

2.5.1 The plant block

The plant is simply our nonlinear model. We wrap `cartpole_dynamics.m` in a thin MATLAB Function block called **plant** that takes the current state $\mathbf{x}$ and the input u and returns the state derivative $\dot{\mathbf{x}}$.

plant (MATLAB Function block)

```
function xdot = plant(x,u,M, m, l, b, g)
    xdot = cartpole_dynamics(x,u,M,m,l,b,g);
end
```

2.5.2 Integrator and initial condition

The output of the plant block is $\dot{\mathbf{x}}$, so we feed it into an integrator ($1/s$) to recover the state $\mathbf{x}$. The integrator carries the initial condition of the simulation. For this controller we start close to the upright equilibrium,

$$\mathbf{x}_0 = [0; 0; 0.2; 0],$$

that is the pendulum tilted by 0.2 rad from vertical, with the cart centred and everything at rest. We deliberately do not start from the hanging-down position used in the setup file, because the LQ regulator was designed on the model linearised about $\theta = 0$ and is only valid in a neighbourhood of that point. A linear regulator cannot swing the pendulum up from the bottom, it can only catch it and hold it once it is already near the top, which is exactly what this small initial tilt tests.

2.5.3 Feedback gain

The loop is closed by the gain block $-L\mathbf{u}$, which applies the static state feedback $u = -L\mathbf{x}$ with the constant gain L computed earlier by `lqr_riccati`. The state coming out of the integrator is

multiplied by $-L$ and sent back to the input port of the plant. Because L is a single constant matrix, the whole regulator is this one block, with no online optimisation during the simulation.

The state is also split by a demux into its four components x , $\dot{x}$, θ and $\dot{\theta}$, which are sent to a scope and saved to the workspace (`out.cart_x` and `out.theta`) for plotting.

2.5.4 The CartPend visualisation block

The **CartPend** block, shown in Figure 2, is a Simscape Multibody model used only to animate the motion. It does no dynamics and no control; it receives the already-computed cart position and pendulum angle and moves a 3D cart and pole accordingly, so we can see the behaviour rather than only read the plots.

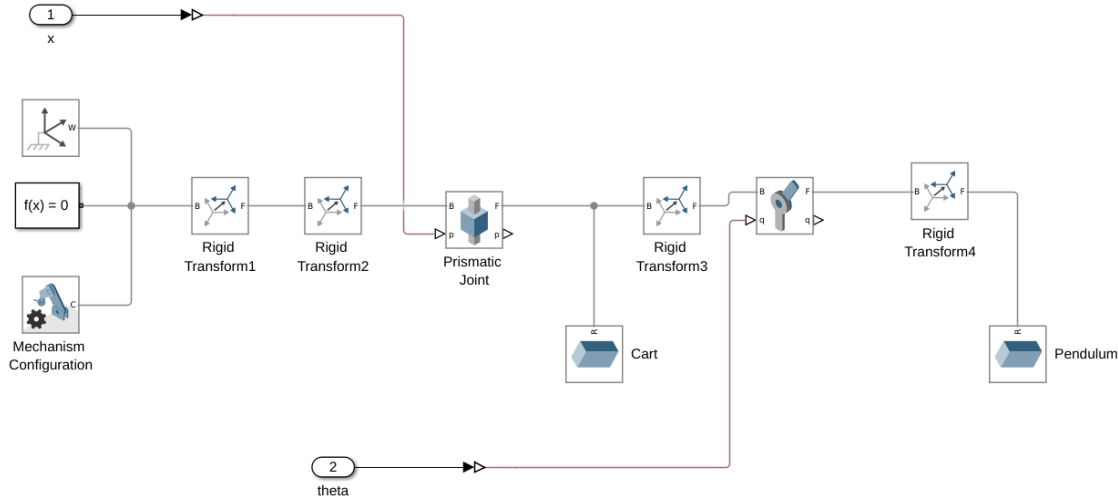


Figure 2: The **CartPend** visualisation subsystem. The two inputs x and θ drive a prismatic joint (the cart sliding horizontally) and a revolute joint (the pendulum rotating), built from rigid transforms and two solid bodies. It is position-driven and computes no dynamics.

Inside the block, the input x (port 1) drives the prismatic joint that lets the cart slide horizontally, and the input θ (port 2) drives the revolute joint that rotates the pendulum. The rigid transforms place the joints and bodies in the right positions, the mechanism configuration sets gravity, and the solver configuration closes the Simscape network. Since both joints are commanded directly from the simulated states, the animation is purely a visual mirror of the numbers produced by the plant and the integrator.

2.5.5 Results

Running the closed loop from the small initial tilt, the LQ regulator brings the pendulum back to the upright position and settles the cart, as shown in Figure 3. The four states x , $\dot{x}$, θ and $\dot{\theta}$ all return to zero, confirming that the gain stabilises the system about the upright equilibrium.

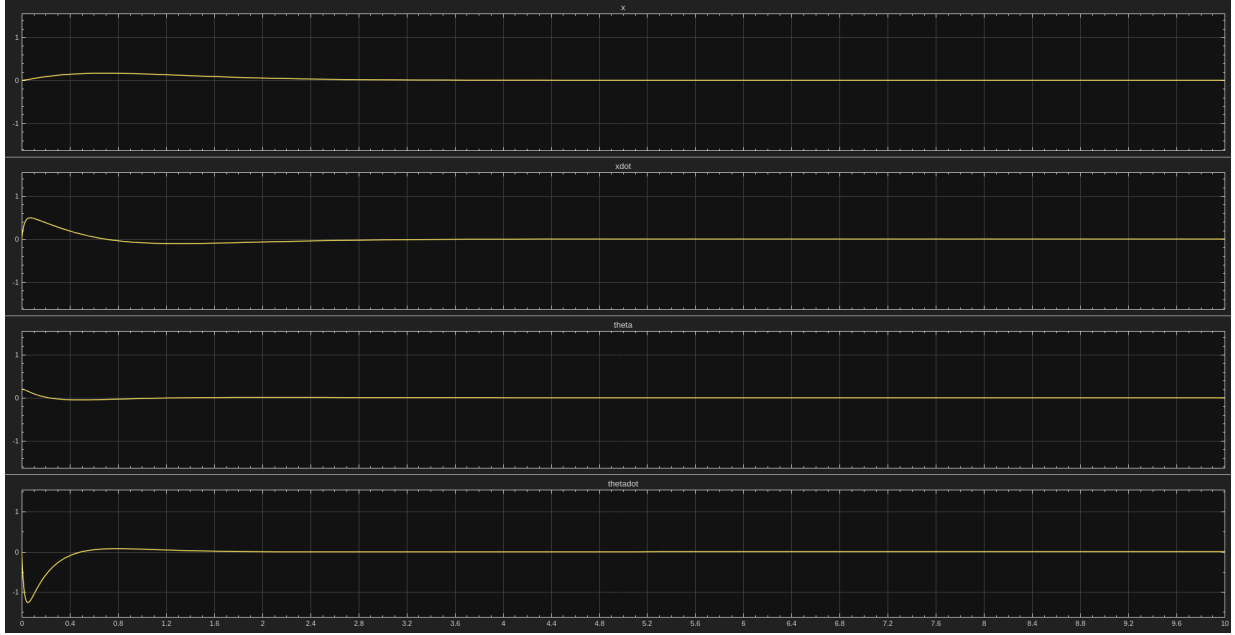


Figure 3: Closed-loop response of the LQ regulator from $\mathbf{x}_0 = [0; 0; 0.2; 0]$: cart position x , cart velocity $\dot{x}$, pendulum angle θ and angular velocity $\dot{\theta}$ against time, all converging to zero.

3 Second Controller: LQ with an Extended Kalman Filter

In the first controller we assumed the full state was available. On the real system this is not true: we only measure the pose, the cart position x and the pendulum angle θ , while the two velocities $\dot{x}$ and $\dot{\theta}$ are never measured directly. To close the loop we therefore need to estimate the missing states, and for this we add an extended Kalman filter (EKF). The result is the LQ + EKF configuration: the same LQ gain as before, but fed with an estimate of the state instead of the true state. The controller itself does not change, what changes is where its input comes from. The full model is shown in Figure 4, and the following sections go through each new piece in detail. Here we only give the general picture of what was added.

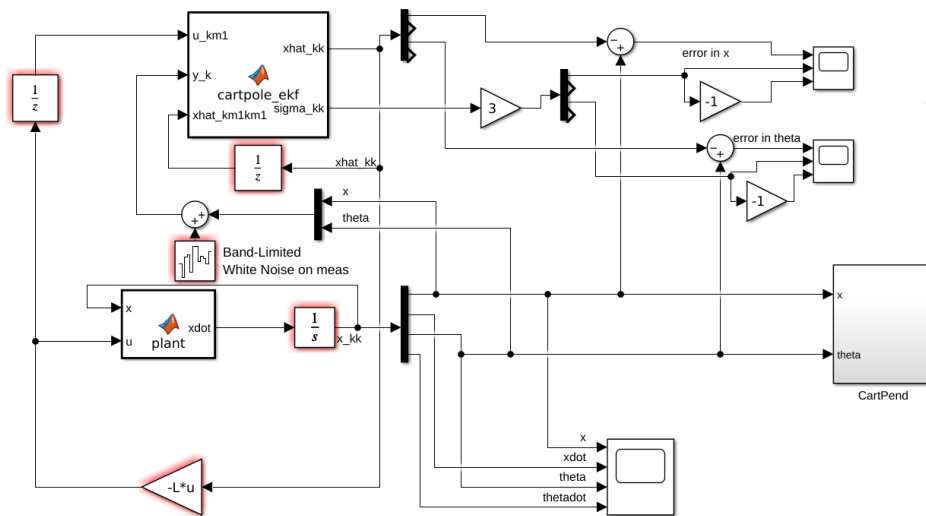


Figure 4: LQ + EKF model. The `cartpole_ekf` block estimates the state from the noisy measurement and the previous input, the $-L \cdot u$ gain now feeds back the estimate $\hat{\mathbf{x}}_{k|k}$, and the scopes at the top compare the estimate against the true state.

3.1 Overview of the changes

The plant, the integrator and the feedback gain are kept exactly as in the previous model. The plant block still holds the nonlinear dynamics, the $1/s$ integrator still produces the true state, and the regulator is still the single $-L \cdot u$ gain. The one change in the loop is that this gain now multiplies the estimated state $\hat{\mathbf{x}}_{k|k}$ coming out of the filter, not the true state, so the controller acts on what it can actually reconstruct.

The new element is the `cartpole_ekf` block. It is a MATLAB Function block that takes the previous input u_{k-1} , the current measurement $\mathbf{y}_k$ and the previous estimate $\hat{\mathbf{x}}_{k-1|k-1}$, and returns the updated estimate $\hat{\mathbf{x}}_{k|k}$ together with its covariance $\Sigma_{k|k}$. The internals of this block, the predict and update steps, are covered in the next section.

The filter is discrete and recursive, so at each step it needs values from the step before. These are provided by the unit delay blocks ($1/z$). One delays the input to give u_{k-1} , and one delays the estimate to give $\hat{\mathbf{x}}_{k-1|k-1}$ both parametrised with a sample time T_s (same as defined in the setup), rather than a hard-coded value.

To test the estimator under realistic conditions, the measurement is not taken clean. The pose $[x; \theta]$ read from the true state is corrupted by a band-limited white noise block before it reaches the filter, so $\mathbf{y}_k$ is a noisy measurement. This lets us check that the EKF still produces a usable estimate when the sensors are imperfect, rather than only in the ideal noise-free case.

The scopes at the top of the model are there to evaluate the filter. They compare the estimate against the true state and display the estimation error in x and in θ , together with the bounds drawn from the filter covariance. These let us judge whether the filter is consistent, and we return to them when we discuss the EKF results.

On the solver side, we tried several of the MATLAB solvers in the model configuration before settling on a variable-step solver with `ode15s`, which handles the mix of the continuous plant and the discrete filter well. Simulink also offers an option to automatically handle rate transitions for data transfer between blocks running at different rates. It can be convenient in mixed continuous and discrete models, but we are not using it here and instead manage the timing explicitly through the unit delays and the shared sample time.

3.2 Inside the `cartpole_ekf` block

The estimator is implemented in the MATLAB Function block `cartpole_ekf`. It is a standard extended Kalman filter built around two stages: a prediction that pushes the previous estimate forward through the model, and an update that corrects it with the new measurement. The code itself is short, so here we focus on the equations and what each one represents. Throughout, $\hat{\mathbf{x}}_{k-1|k-1}$ is the estimate from the previous step, $\hat{\mathbf{x}}_{k|k-1}$ the prediction before seeing the measurement, and $\hat{\mathbf{x}}_{k|k}$ the corrected estimate; the matrices Σ with the same subscripts are the corresponding state covariances.

3.2.1 Tuning matrices

The filter needs three covariance matrices, set once at the top of the block:

$$Q = \text{diag}(10^{-5}, 10^{-4}, 10^{-5}, 10^{-4}), \quad R = \text{diag}(2.5 \times 10^{-3}, 2.5 \times 10^{-3}), \quad P_0 = \text{diag}(5, 5, 5, 5).$$

Q is the process noise covariance and expresses how much we trust the model: the smaller its entries, the more the filter leans on the dynamics rather than on the measurements. We keep it small, with slightly larger values on the two velocities since those are the states the model predicts least reliably. R is the measurement noise covariance; its two entries are the variances of the position and angle measurements. P_0 is the initial state covariance, set large so the filter starts with high uncertainty and relies on the first measurements while it converges. The covariance is stored in a persistent variable so it carries from one step to the next, initialised to P_0 on the first call, while the previous estimate is fed back into the block through the unit delay.

3.2.2 Prediction step

The prediction propagates the previous estimate through the nonlinear dynamics f (the same `cartpole_dynamics` used everywhere) over one sample T_s . We do this with a fourth-order Runge-Kutta (RK4) step rather than a single Euler step, because RK4 follows the curved continuous trajectory much more accurately over the interval:

$$\begin{aligned} k_1 &= f(\hat{\mathbf{x}}_{k-1|k-1}, u_{k-1}), & k_2 &= f\left(\hat{\mathbf{x}}_{k-1|k-1} + \frac{T_s}{2}k_1, u_{k-1}\right), \\ k_3 &= f\left(\hat{\mathbf{x}}_{k-1|k-1} + \frac{T_s}{2}k_2, u_{k-1}\right), & k_4 &= f\left(\hat{\mathbf{x}}_{k-1|k-1} + T_s k_3, u_{k-1}\right), \\ \hat{\mathbf{x}}_{k|k-1} &= \hat{\mathbf{x}}_{k-1|k-1} + \frac{T_s}{6}(k_1 + 2k_2 + 2k_3 + k_4). \end{aligned}$$

To propagate the covariance we need the sensitivity of this map to the state, that is its Jacobian F . Instead of differentiating the RK4 step, we linearise the simpler Euler one-step map $\phi(\mathbf{x}) = \mathbf{x} + T_s f(\mathbf{x}, u_{k-1})$, whose Jacobian is $F = I + T_s \partial f / \partial \mathbf{x}$. We evaluate it numerically by central differences, perturbing each state component by a small ε :

$$F_{:,i} = \frac{\phi(\hat{\mathbf{x}}_{k-1|k-1} + \varepsilon e_i) - \phi(\hat{\mathbf{x}}_{k-1|k-1} - \varepsilon e_i)}{2\varepsilon}, \quad i = 1, \dots, 4,$$

with e_i the i -th unit vector. Central differences give a more accurate Jacobian than a one-sided difference at little extra cost, and they save us from deriving $\partial f / \partial \mathbf{x}$ by hand. The predicted covariance is then

$$\Sigma_{k|k-1} = F \Sigma_{k-1|k-1} F^\top + Q.$$

Here F is this local Jacobian, recomputed at every step from the current estimate; it is not the constant discrete matrix F of the LQ design, even though we reuse the letter to match the code.

3.2.3 Update step

The update corrects the prediction with the new measurement $\mathbf{y}_k$. Because the measurement model is linear, $\mathbf{y} = G\mathbf{x}$ with G the same pose-selection matrix introduced earlier, the predicted measurement and the innovation covariance are

$$\hat{\mathbf{y}}_{k|k-1} = G \hat{\mathbf{x}}_{k|k-1}, \quad S_k = G \Sigma_{k|k-1} G^\top + R.$$

S_k is the covariance of the innovation $\mathbf{y}_k - \hat{\mathbf{y}}_{k|k-1}$, the gap between what we measured and what we expected. The Kalman gain weighs the prediction against the measurement,

$$H_k = \Sigma_{k|k-1} G^\top S_k^{-1}$$

and the corrected estimate and covariance follow as

$$\begin{aligned} \hat{\mathbf{x}}_{k|k} &= \hat{\mathbf{x}}_{k|k-1} + H_k (\mathbf{y}_k - \hat{\mathbf{y}}_{k|k-1}), & \Sigma_{k|k} &= \Sigma_{k|k-1} - H_k S_k H_k^\top. \\ \sigma_{k|k} &= \sqrt{\text{diag}(\Sigma_{k|k})}, \end{aligned}$$

3.2.4 Linearising only the dynamics

A full EKF linearises both the dynamics and the measurement at each step, but here only the dynamics are nonlinear, so only F has to be computed. The measurement $\mathbf{y} = G\mathbf{x}$ is already linear in the state, which means its Jacobian is exactly the constant matrix G . There is nothing to linearise on the measurement side, and G is used directly in S_k and H_k . This is why the numerical Jacobian is needed only for the prediction and not for the output.

3.3 Measurement noise

To test the filter we corrupt the measured pose with the band-limited white noise block, with the noise power set to $[2.5 \times 10^{-6}, 2.5 \times 10^{-6}]$, one value for each measured channel, the position x and the angle θ . We add this noise on the measured outputs and not on the state itself, adding the noise anywhere else would not make sense physically.

3.4 Closed-loop response

Figure 5 shows the four states over the ten-second run. The closed loop stabilises: the cart position x and the pendulum angle θ settle smoothly to zero, and the two velocities $\dot{x}$ and $\dot{\theta}$ return to zero as well. Once the system reaches the upright equilibrium the states keep a small residual jitter around zero, visible mostly on $\dot{\theta}$, which comes from the injected measurement noise. The amplitude stays minimal and does not disturb the regulator.

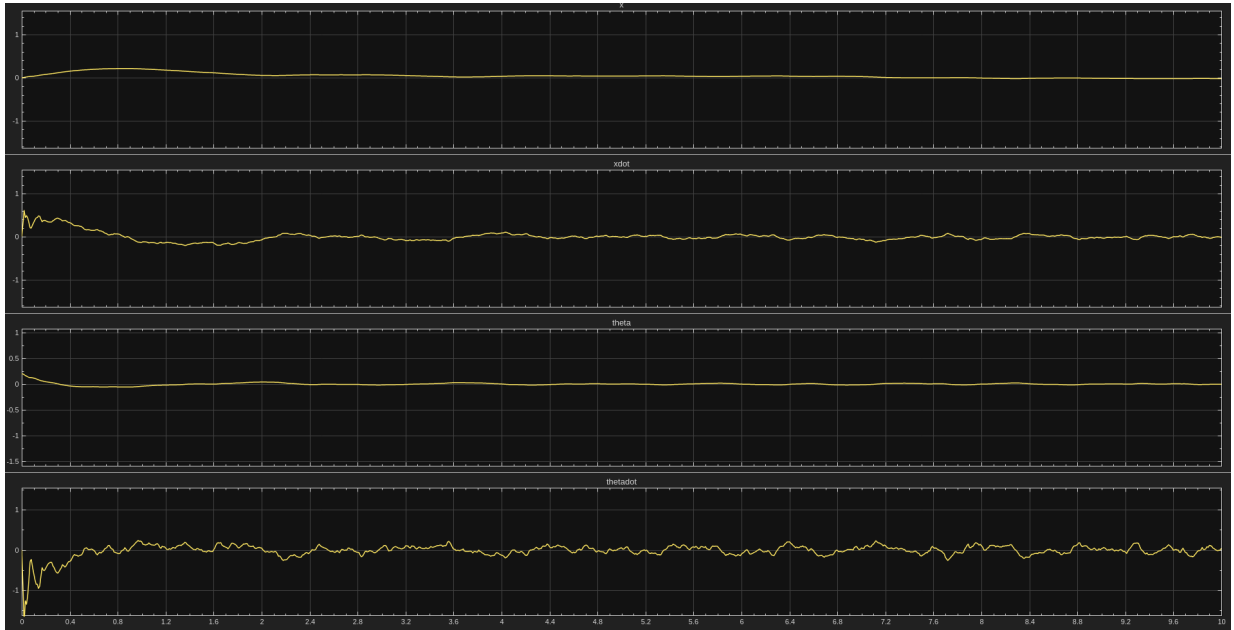


Figure 5: Closed-loop response of the LQ + EKF controller: the four states x , $\dot{x}$, θ and $\dot{\theta}$ over the run. All settle to zero, with a small residual jitter from the measurement noise, most noticeable on $\dot{\theta}$.

3.5 Filter consistency: the three-sigma bands

To judge the filter itself we plot the estimation error against the three-sigma confidence band built from the EKF covariance, that is $\pm 3\sigma_{k|k}$. Figure 6 shows the error in x and in θ in blue, together with these bands in red. In both cases the band starts wider during the initial transient and then settles to a constant width as the covariance converges. The estimation error stays inside the $\pm 3\sigma$ band for the whole run, which means the filter is not underestimating its own uncertainty, and the error fluctuates close to zero with a small, non-severe amplitude. Together these indicate a consistent and accurate state estimate.

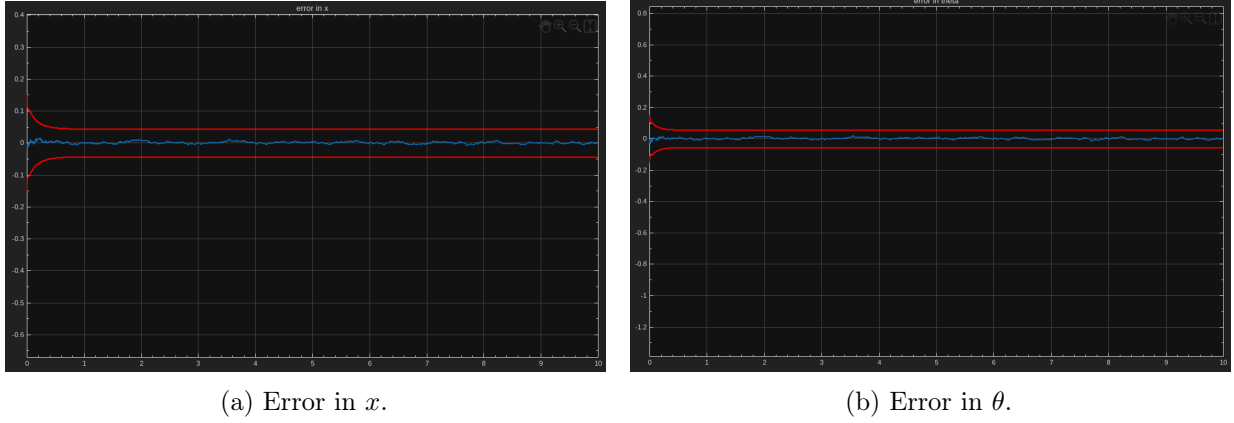


Figure 6: Estimation error (blue) against the $\pm 3\sigma$ band from the filter covariance (red). The error stays inside the band throughout and fluctuates close to zero, so the filter is consistent and the estimate is accurate.

4 Final Controller: Swing-Up plus LQ + EKF

The LQ + EKF controller can only stabilise the pendulum once it is already near the upright position, because the gain was designed on the model linearised about $\theta = 0$. To start from the natural hanging-down position we first have to swing the pendulum up to the top, and only then hand control over to the LQ + EKF regulator. This final configuration adds a swing-up controller and a switching logic on top of the previous model: we swing up, and once the pendulum is close enough to upright and slow enough, we switch to the LQ + EKF controller, which catches and holds it.

For the swing-up we chose a finite-horizon (FH) open-loop controller. The optimal input is computed offline once and stored as a time-indexed sequence $u(t)$, which is then played back during the swing-up phase. In this combined model we run the LQ + EKF part without injecting measurement noise, so that we can focus on the swing-up and the handover rather than on the filter's noise rejection, which was already studied in the previous section.

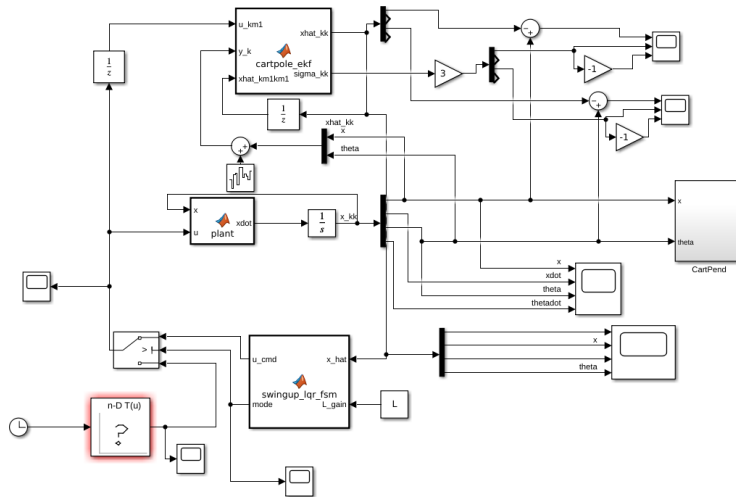


Figure 7: Swing-up plus LQ + EKF model. The previous estimation and regulation blocks are unchanged; the new parts are the lookup table holding the offline swing-up input $u(t)$, driven by a clock, and the `swingup_lqr_fsm` block that switches between the swing-up input and the LQ feedback.

4.1 Overview of the changes

The first new entity is the offline-computed swing-up input stored in a lookup table. The FH controller produces an optimal input sequence $u(t)$, which is loaded into a lookup table indexed by time (the `n-D T(u)` block driven by a clock). During the swing-up phase this table simply plays back the precomputed input, so no optimisation runs online.

The second new entity is the switching logic, implemented as the MATLAB Function block `swingup_lqr_fsm`. It is a small finite-state machine that receives the state estimate $\hat{x}$ and the LQ gain L , and decides which controller is active.

The setup file must also be updated. The finite-horizon section of `cartpole_setup.m`, previously commented out, is now uncommented so that the swing-up input is built before the simulation runs:

```
cartpole_setup.m (finite-horizon swing-up)

%% FINITE HORIZON SET UP
Ts_swing = 0.02;
[u_opt, x_pred] = fh_controller(x0, M, m, l, b, g, Ts_swing);
t_vec = (0:length(u_opt)-1)' * Ts_swing;
% Save it to the workspace
assignin('base', 't_swing_lut', t_vec);
assignin('base', 'u_swing_lut', u_opt);
```

This block calls `fh_controller` to compute the optimal open-loop input u_{opt} that swings the pendulum up from the initial hanging-down state x_0 , using a coarser swing-up sample time $T_{s,\text{swing}} = 0.02$ s. It then builds the matching time vector t_{vec} and writes the pair `t_swing_lut`, `u_swing_lut` to the base workspace, which is exactly what the lookup table block reads. With this in place, the swing-up trajectory is ready before the model starts.

4.2 The finite-horizon swing-up problem

The swing-up input is found by `fh_controller`, which solves a finite-horizon optimal control problem offline. We discretise the horizon into N steps of length T_s and write $\Phi(x, u)$ for the map that integrates the nonlinear dynamics over one interval with a fixed-step RK4 rule (using ten substeps per interval for accuracy, implemented in `step_rk4`). The decision variables are both the input sequence and the state sequence,

$$z = [u_0, \dots, u_{N-1}, x_1, \dots, x_N],$$

and the problem solved is

$$\min_z \sum_{k=0}^{N-1} \left(x_k^\top V x_k + u_k^\top P u_k \right) + x_N^\top V_N x_N$$

subject to

$$\begin{aligned} x_{k+1} - \Phi(x_k, u_k) &= 0, & k &= 0, \dots, N-1, \\ |x_k^{(1)}| &\leq x_{\text{track}}, & k &= 1, \dots, N, \\ |u_k| &\leq u_{\text{max}}, & k &= 0, \dots, N-1, \end{aligned}$$

with x_0 fixed. The first line is the set of dynamics defects, the equality constraints that force the guessed states to obey the physics. The second keeps the cart position $x_k^{(1)}$ inside the track limit $x_{\text{track}} = 0.8$ m, and the third saturates the force at $u_{\text{max}} = 20$ N. The cost has the same structure as the LQ cost: a stage weight $V = 2 \text{diag}(20, 5, 10, 5)$ on the states, an input weight $P = 0.1$,

and a large terminal weight $V_N = \text{diag}(5000, 500, 1000, 500)$ that strongly pulls the final state to the upright equilibrium. The cost lives in `ms_cost`, the defects and track limits in `ms_con`, and the whole problem is handed to `fmincon` with the SQP algorithm. The function returns only the optimal input `u_opt`; the optimal states are used internally and then discarded.

4.3 From single shooting to multiple shooting

The formulation above, where the states are decision variables and the dynamics are equality constraints, is called multiple shooting. We did not start there. The first version was a single-shooting problem: the only decision variables were the inputs, and the whole state trajectory was obtained by forward-integrating from x_0 . That version swung the pendulum up correctly from the hanging-down position, but it failed near upright. Starting from a small tilt such as $[0, 0, 0.1, 0]$, the planned angle whipped out to about 4 rad and back, with ± 20 N input and the cart slamming into the track limit.

The reason is that the upright equilibrium is unstable, so the forward rollout is exponentially sensitive to the inputs and the cost surface is full of local minima. With a near-zero initial guess for the input, the rollout already had the pole falling and swinging around before the optimiser did anything, so the solver searched inside that swing-around basin and only refined it, reaching a cost of about 25 000 while the gentle hold solution scored about 52. The true answer existed, but the solver simply never found it.

Multiple shooting removes this problem. Because the optimiser now shapes the state trajectory directly and the dynamics are enforced as defects $x_{k+1} - \Phi(x_k, u_k) = 0$, a trivial straight-line guess converges to the true optimum from any starting point, while still being open loop.

4.4 Choosing the horizon and the sample time

Two numbers shape the problem: the number of steps N and the sample time T_s . Their product $N T_s$ is the total planning horizon, here $100 \times 0.02 = 2$ s. The horizon must be long enough to physically perform the manoeuvre and settle at the top. If it is too short there is not enough time and the plan becomes infeasible or unnecessarily violent, while if it is too long the solve is heavier and the trajectory can diverge. The sample time sets the resolution of the input and state grid: a smaller T_s gives a finer, smoother input and a more faithful representation of the dynamics, but more variables and a slower solve, whereas a coarser T_s is cheaper but can miss fast motion and weaken the defect constraints. In practice we pick T_s fine enough to resolve the fastest part of the swing and keep the input smooth, then choose N so that $N T_s$ covers a realistic swing-up duration of one to two seconds. The ten RK4 substeps per interval keep the integration accurate even though T_s is moderately coarse.

4.5 The optimal swing-up trajectory

Solving the problem from the hanging-down state produces the open-loop trajectory in Figure 8, which is the behaviour the system is planned to follow. The angle θ leaves π and decreases smoothly to zero in about 0.8 s, then stays there. To pump energy into the pole the cart first moves out to the track limit at -0.8 m and then returns to the centre by about 1.4 s, with the cart velocity and angular velocity peaking during the swing and decaying back to zero. The input saturates near ± 20 N early on, where most of the energy is injected, and then relaxes to almost nothing once the pole is up. The plan respects both constraints throughout: the cart position never exceeds the limit and the force stays within saturation. This precomputed sequence is what the lookup table plays back, after which the LQ + EKF controller takes over to hold the pendulum upright.

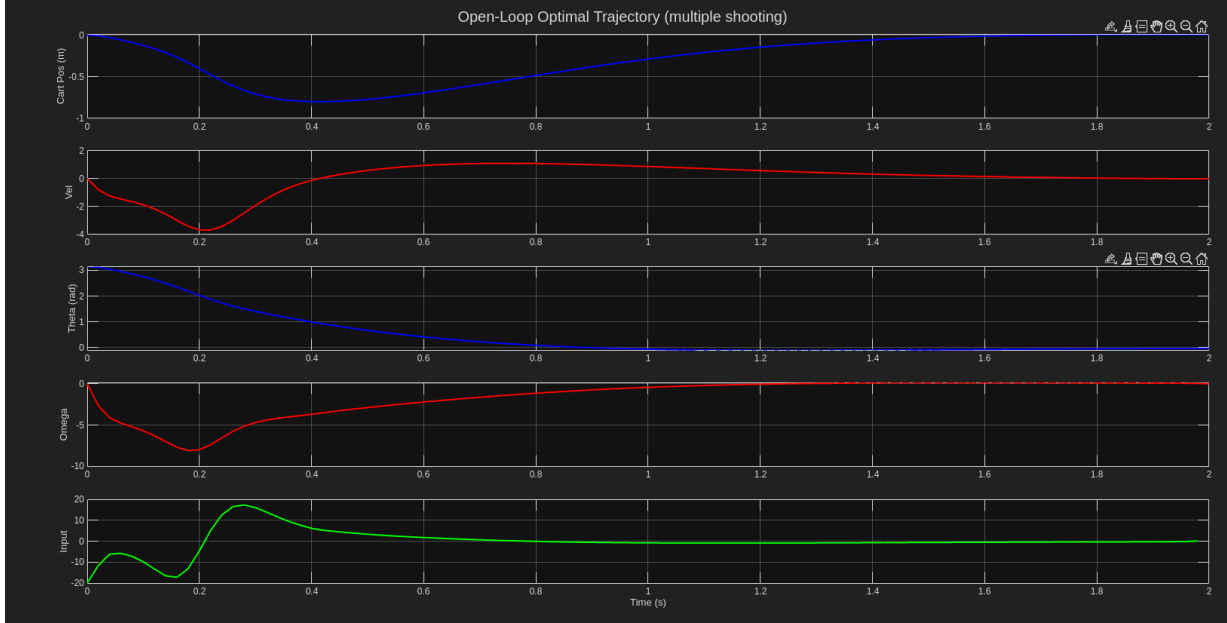


Figure 8: Open-loop optimal swing-up trajectory from the hanging-down state, found by multiple shooting. The angle rises from π to 0, the cart moves to the track limit and back, and the input saturates early then relaxes. All constraints are respected.

4.6 The catch-and-hold supervisor

The decision of when to hand control from the swing-up to the regulator is made by `swingup_lqr_fsm`, a small finite-state machine with two modes. In mode 0 the swing-up drives the force, with the open-loop sequence played back from the lookup table; in mode 1 the LQ + EKF regulator drives the force. The block outputs this mode, which is wired to the control input of the force switch, so the mode directly selects which signal reaches the plant. It also computes and outputs the regulator command itself.

The supervisor receives the state estimate $\hat{\mathbf{x}}$ and the gain L . It first wraps the estimated angle to $[-\pi, \pi]$ with $\theta \leftarrow \text{atan2}(\sin \theta, \cos \theta)$, so that the comparison to the upright position is correct even if the pole has turned through several revolutions during the swing. The mode then follows a simple rule with hysteresis:

$$\begin{aligned} \text{mode } 0 &\rightarrow 1 && \text{when } |\theta| < \theta_{\text{catch}} \text{ and } |\omega| < \omega_{\text{catch}}, \\ \text{mode } 1 &\rightarrow 0 && \text{when } |\theta| > \theta_{\text{fall}}, \end{aligned}$$

with $\theta_{\text{catch}} = 0.15$ rad, $\omega_{\text{catch}} = 1.5$ rad/s and $\theta_{\text{fall}} = 0.8$ rad. The catch condition is deliberately strict: the regulator is allowed to take over only when the pole is both close to upright and moving slowly, which is exactly the region where the linear design is valid. The release condition is much wider, so once the regulator has caught the pole it keeps control unless the angle grows past 0.8 rad. The gap between the two thresholds stops the machine from chattering between modes near the boundary, and the release acts as a safety fallback that should not trigger in normal operation. The mode is held in a persistent variable so it carries between steps.

At every step the block also forms the regulator command $u = -L \hat{\mathbf{x}}$ and saturates it to $\pm F_{\text{max}}$ with $F_{\text{max}} = 20$ N, matching the swing-up force limit. This command is the signal the switch passes in mode 1.

4.7 Results of the full controller

Running the complete controller from the hanging-down position gives the response in Figure 9. The swing-up phase brings the angle θ from π down to zero in about one second, closely following

the planned open-loop trajectory: the cart first moves out toward the track limit and back, and the cart and angular velocities show the same early dips before returning to zero. As the pole reaches the top slowly enough, the supervisor switches to LQ mode and the regulator catches and holds it. From there all four states settle to zero and stay there, with no fall-back to swing-up and no visible discontinuity at the handover. Because no measurement noise is injected in this run, the response is clean, which makes the swing-up and the catch easy to read. This confirms that the three pieces work together as intended: the finite-horizon plan swings the pole up, the supervisor decides when the catch is safe, and the LQ + EKF regulator stabilises the upright equilibrium.

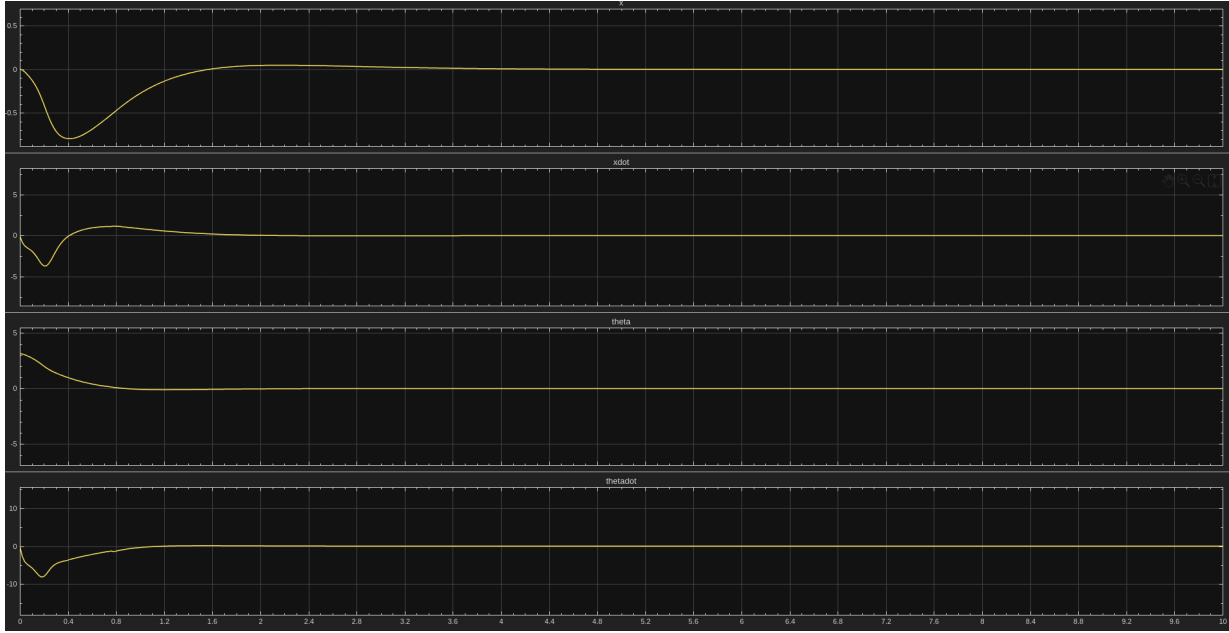


Figure 9: Closed-loop response of the full swing-up plus LQ + EKF controller from the hanging-down state. The angle rises from π to 0 during the swing-up, the supervisor hands over near the top, and the LQ + EKF regulator holds all four states at zero.

4.8 Comparing the plan with the real run

As a last check we compare the offline plan against what actually happened in the simulation. A short post-processing script reads the logged signals from `out`, overlays the planned trajectory on the simulated one, and computes the tracking error in the workspace after the run. The error is evaluated only during the swing-up phase, while the mode is 0, because once the regulator catches the pole it drives the force directly and the open-loop plan no longer applies.

Figure 10 shows the result. On the left the planned states (blue) and the actual states (red) are plotted together, and they sit almost on top of each other through the whole swing-up, with the dashed line marking the catch. On the right is the tracking error over the swing-up window, from zero to about 0.76 s. The deviations stay very small throughout: the position error is at most a few millimetres, the angle error a few thousandths of a radian, and the velocity and angular-velocity errors a few hundredths in their units. They grow slightly toward the end of the swing-up as small sampling and estimation differences accumulate, but never become significant. The bottom panel shows the mode, which stays in swing-up until about 0.76 s and then switches to LQR for the rest of the run.

The closeness of the two trajectories confirms that the real closed loop follows the precomputed open-loop plan faithfully during the swing-up, and that the supervisor hands over at the right moment.

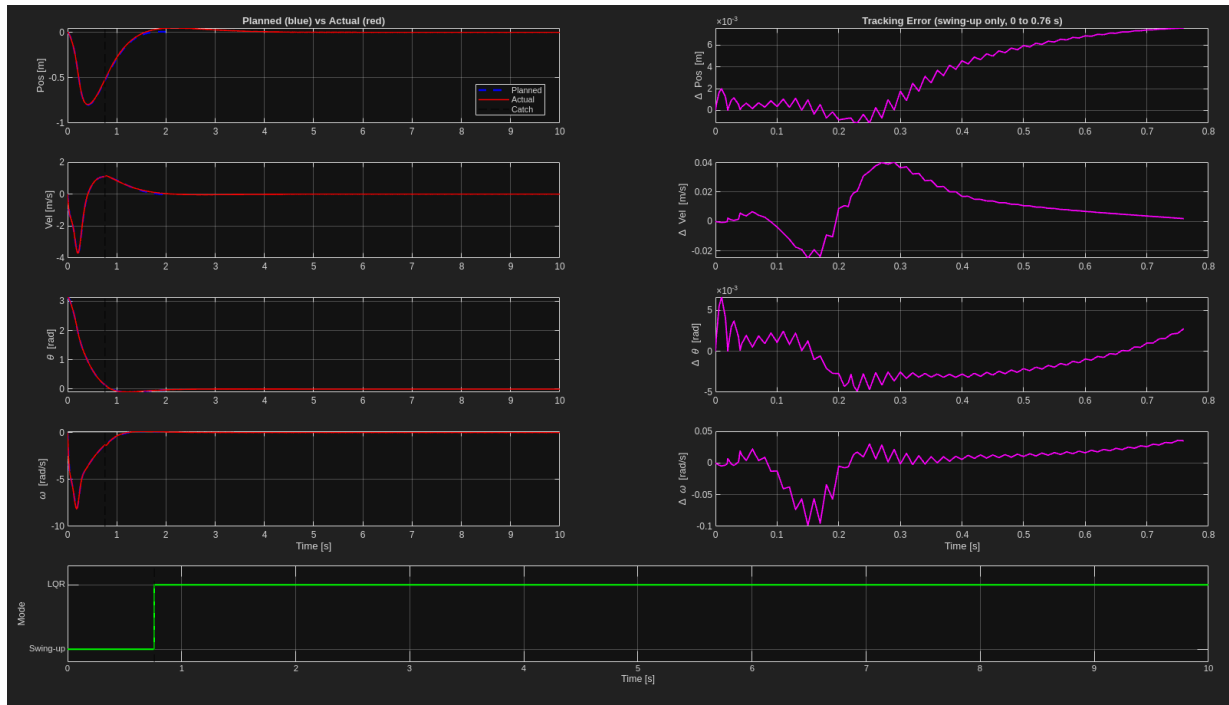


Figure 10: Planned (blue) versus actual (red) trajectory during the swing-up, with the tracking error on the right and the mode at the bottom. The actual run tracks the plan closely, the errors stay small, and the supervisor switches from swing-up to LQR at about 0.76 s.